CS336 Assignment 2: Systems and Parallelism

Kesavan Ramakrishnan

May 7, 2026

2 Profiling and Benchmarking

2.1 Problem (`benchmarking_script`)

(b) Timings across model sizes

Table 1: Forward, backward, and optimizer-step timings (ms, mean ± std over 10 iters, 5 warmup) on B200 in FP32.

size	forward	backward	optim
small	16.86 ± 0.04	32.12 ± 0.02	7.63 ± 0.02
medium	47.80 ± 0.13	92.78 ± 0.09	17.14 ± 0.12
large	106.85 ± 0.06	208.42 ± 0.18	30.53 ± 0.14
xl	305.85 ± 0.14	571.76 ± 0.39	80.32 ± 0.16

Variability across runs isn't very high, the standard deviation is low and numbers are pretty stable.

(c) Effect of warm-up steps

Table 2: **No Warmup**: Forward, backward, and optimizer-step timings (ms, mean ± std over 10 iters, 5 warmup).

size	forward	backward	optim
small	33.98 ± 51.47	35.51 ± 10.47	7.87 ± 0.13
medium	47.91 ± 0.84	92.57 ± 0.11	16.66 ± 0.11
large	109.19 ± 8.00	208.14 ± 0.39	29.86 ± 0.20
xl	317.26 ± 69.70	570.19 ± 0.55	79.74 ± 0.23

Table 3: **2 iter Warmup**:Forward, backward, and optimizer-step timings (ms, mean ± std over 10 iters, 5 warmup) on B200 in FP32.

size	forward	backward	optim
small	16.74 ± 0.08	32.02 ± 0.03	7.32 ± 0.03
medium	47.50 ± 0.05	92.41 ± 0.04	16.51 ± 0.04
large	106.50 ± 0.06	207.61 ± 0.09	29.87 ± 0.18
xl	295.88 ± 0.23	569.12 ± 0.13	80.85 ± 0.10

With no warmup steps the results are much noisier because the GPU has to warmup and there are latencies that come up from compiling the kernels, kernel setup, etc, making the first runs much longer. As we can see, with no warmup the reported latencies are much higher and have much more variability, but as soon as even 2 warmups are used, the numbers return to normal.

2.2 Problem (`nsys_profile`)

Table 4: Nsight Systems profiling configurations.

model size	context length	batch size	profiled mode
small	256	4	train
small	1024	4	train
small	2048	4	train
large	256	4	train
large	1024	4	train
large	2048	4	train

(a) Total forward pass time vs. Python timing

Table 5: Forward-pass runtime from Nsight Systems compared with Python timing.

profile	nsys fwd (ms)	Python fwd (ms)	difference
small, ctx 256	8.634	10.17	1.536
small, ctx 1024	16.880	37.46	20.58
small, ctx 2048	18.582	90.62	72.038
large, ctx 256	48.389	62.58	14.191
large, ctx 1024	103.621	229.86	126.239
large, ctx 2048	323.612	521.60	197.988

The numbers do not align but this makes sense because the nsys NVTX forward range times the CPU queue window and so it doesn't wait for all the kernels to finish. However, for python side timing, I include a torch.cuda.synchronize() after the forward call and then end timing, which

means the Python side timing measures the total kernel time, which is why the python side timing is higher than the nsys one.

(b) Most expensive CUDA kernel in the forward pass

Table 6: CUDA kernels with the largest cumulative GPU time.

profile	dominant fwd kernel	fwd calls	dominant full-step kernel	full-step calls
small, ctx 256	`cutlass3x_sm100_sgemm_64x64x16_tnn_relu`	24	`cutlass3x_sm100_sgemm_128x64x16_nnn_relu`	72
small, ctx 1024	`cutlass3x_sm100_sgemm_64x64x16_tnn_relu`	28	`cutlass3x_sm100_sgemm_64x64x16_tnn_relu`	28
small, ctx 2048	`cutlass3x_sm100_sgemm_128x128x16_tnn_relu`	7	`cutlass3x_sm100_sgemm_128x128x16_tnn_relu`	39
large, ctx 256	`cutlass3x_sm100_sgemm_64x32x16_tnn_relu`	139	`cutlass3x_sm100_sgemm_64x32x16_nnn_relu`	223
large, ctx 1024	`cutlass3x_sm100_sgemm_128x128x16_tnn_relu`	49	`cutlass3x_sm100_sgemm_64x64x16_ntn_relu`	212
large, ctx 2048	`cutlass3x_sm100_sgemm_128x128x16_tnn_relu`	49	`cutlass3x_sm100_sgemm_128x128x16_tnn_relu`	135

The dominant forward-pass CUDA kernels are all CUTLASS SGEMM kernels. For the shorter-context cases this is a matmul that comes from attention, while the longer-context / larger-model creates wider FFN-shaped GEMMs. The dominant full forward+backward kernel is sometimes the same tile as the forward pass, which indicates that a backward GEMM such as a weight-gradient or activation-gradient matmul is taking the most cumulative GPU time.

(c) Non-matmul kernels with non-trivial runtime

Table 7: Non-matmul CUDA kernels with non-trivial forward-pass runtime.

profile	non-matmul kernels observed	likely component
small, ctx 256	`aten_elementwise_div`	attention + FFN elementwise kernels
small, ctx 1024	`aten_elementwise_div`	attention + FFN elementwise kernels
small, ctx 2048	`aten_elementwise_div`	attention + FFN elementwise kernels
large, ctx 256	`aten_elementwise_div`	attention + FFN elementwise kernels
large, ctx 1024	`aten_elementwise_div`	attention + FFN elementwise kernels
large, ctx 2048	`aten_elementwise_div`	attention + FFN elementwise kernels

For the forward pass there were a lot of elementwise kernels throughout that came from the attn and ffn operations.

(d) Full training step vs. inference fraction breakdown

Table 8: CUDA runtime fraction by kernel category for inference and full training steps.

category	mode	matmul	elementwise / optim	reductions / softmax	movement / indexing
small, ctx 256	forward	78.3%	13.9%	4.2%	3.2%
small, ctx 256	train	64.2%	26.7%	3.6%	5.0%
small, ctx 1024	forward	66.0%	24.6%	7.0%	2.1%
small, ctx 1024	train	64.8%	28.9%	3.8%	2.3%
small, ctx 2048	forward	56.1%	31.8%	8.0%	1.7%
small, ctx 2048	train	56.8%	36.9%	4.4%	1.5%
large, ctx 256	forward	87.0%	8.6%	2.5%	1.9%
large, ctx 256	train	73.5%	20.7%	2.5%	4.1%
large, ctx 1024	forward	76.7%	17.7%	4.5%	1.4%
large, ctx 1024	train	73.0%	21.5%	2.6%	2.5%
large, ctx 2048	forward	66.2%	24.8%	6.6%	1.2%
large, ctx 2048	train	61.0%	33.3%	3.9%	1.8%

Categories: matmul is the CUTLASS SGEMM kernels; elementwise/optim includes add, multiply, divide, sigmoid, pow, sqrt, fill, and AdamW-style vectorized kernels; reductions/softmax comes from reduce max/sum/mean, exp/log, rsqrt, and softmax-related kernels; movement/indexing includes cat/copy, arange, scatter/gather, and indexing kernels.

Generally in comparison to just the forward pass, the full train step is still dominated by matmuls but elementwise and optimizer ops take up a larger percentage in the full train step, making the matmul a smaller percent than in the forward.

(e) Softmax vs. matmul runtime within self-attention

Table 9: Runtime comparison of softmax and matrix multiplication inside self-attention.

profile	softmax (ms)	mask (ms)	scores matmul (ms)	output matmul (ms)	matmul total (ms)	softmax/matmul
small, ctx 256	1.120	0.563	1.011	0.904	1.915	0.58
small, ctx 1024	1.138	0.586	1.029	0.953	1.982	0.57
small, ctx 2048	1.172	0.575	1.010	0.933	1.943	0.60
large, ctx 256	1.858	0.904	1.647	1.555	3.202	0.58
large, ctx 1024	28.851	13.033	1.692	1.629	3.321	8.69
large, ctx 2048	56.977	27.134	1.781	1.681	3.462	16.46

Comparing the softmax to the total time for matmuls, we see that the softmax takes a significant amount of time especially at a bigger model size and longer context length where it dominates. Softmax FLOPs is O(n) while matmul FLOPs is O(n^3) which shows that softmax takes significantly more time than difference in FLOPs.

2.3 Problem (`mixed_precision_accumulation`)

When accumulating in a lower precision (f16), we see that the output of the function is 9.9531 which is inaccurate. However, when doing computation in fp32, the value is precise, and when doing an fp16 operation but accumulating in fp32, so you can operate in a lower precision, accuracy is still recovered (10.0021 vs 10.0001), showing that accumulating in a higher precision preserves accuracy even if the operation is done in a lower precision.s

2.4 Problem (`benchmarking_mixed_precision`)

(a) Data types under autocast (FP16)

Model parameters dtype: fp32 Output of first feed-forward: fp16 Output of layer norm: fp32 Model's logits: fp16 Loss: fp32 Gradients: fp32

(b) Layer norm sensitivity in mixed precision

With fp16 mixed precision, the layer norm is done in fp32 because the layer norm calculates the mean and variance of a set of values, which can become a big value that can't be stored in fp16, making it sensitive. If we were to do it in bf16 this would be better since we can represent the range of fp32, but we then lose precision because we have less mantissa bits, meaning it will be less accurate.

(c) BF16 mixed precision benchmarking

Table 10: FP32 versus BF16 mixed-precision timing by model size.

size	FP32 fwd (ms)	BF16 fwd (ms)	FP32 bwd (ms)	BF16 bwd (ms)
small	16.63	8.03	31.80	13.50
medium	47.20	16.53	91.86	31.98
large	105.91	29.76	206.26	59.01
xl	291.92	47.16	567.16	104.68

Table 11: Mixed-precision speedup relative to FP32.

size	forward speedup	backward speedup
small	2.07×	2.36×
medium	2.86×	2.87×
large	3.56×	3.50×
xl	6.19×	5.42×

From looking at the table we see that operating in bf16 results in significantly lower runtimes for both the fwd and bwd passes. On top of that, as model size grows we can see that speedups become even more pronounced.

2.5 Problem (`memory_profiling`)

(a) Memory timeline screenshots

Figure 1: Memory timeline for the XL forward-only profile without autocast.

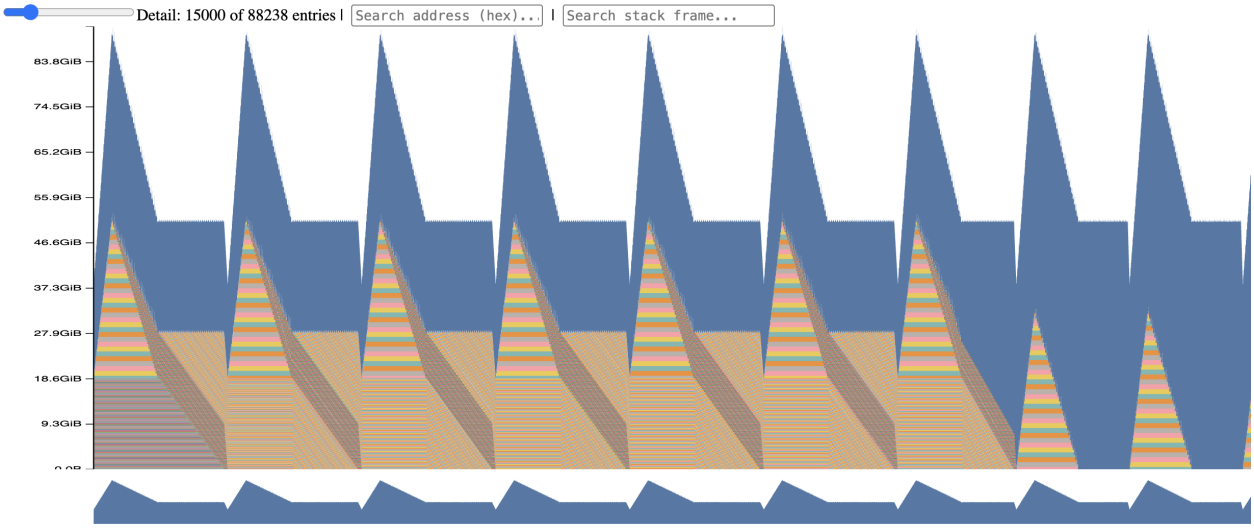


Figure 2: Memory timeline for the XL full training-step profile without autocast.

Looking at these images we can see that the training timeline shows spikes in memory usage due to activations being stored, and then it drops, presumably after an optimizer step. The forward only timeline is flat the entire time as the only memory being used is by the weights being stored in memory.

(b) Peak memory per context length

Table 12: Peak memory usage by context length.

context length	forward peak (MiB)	train peak (MiB)
128	13,154	52,634
2048	15,305	93,356

(c) Peak memory under mixed precision

With mixed precision we see a modest effect on peak memory, because for the forward pass at ctx length = 128, the dominant memory comes from things like parameters, gradients, and optimizer states which aren't effected by autocast. For the training step with ctx length = 2048, we see that peak drops from 93 GiB to 84 GiB, as some activations are now stored as bf16.

(d) Size of an XL residual-stream activation tensor

For each activation tensor we can calculate memory by B * S * d_model * bytes_per_elem.

For context length 128 this is:

1 * 128 * 2560 * 4 = 1,310,720 bytes = **1.25 MiB**

For context length 2048 this is: 1 * 2048 * 2560 * 4 = 20,971,520 bytes = **20MiB**

(e) Largest allocations in the active memory timeline

The largest allocations for the sequence length 2048 was around 2GiB, and we can see that these allocations come during forward passes specifically during the softmax operation inside SDPA.

(f) Memory per TransformerBlock saved for backward

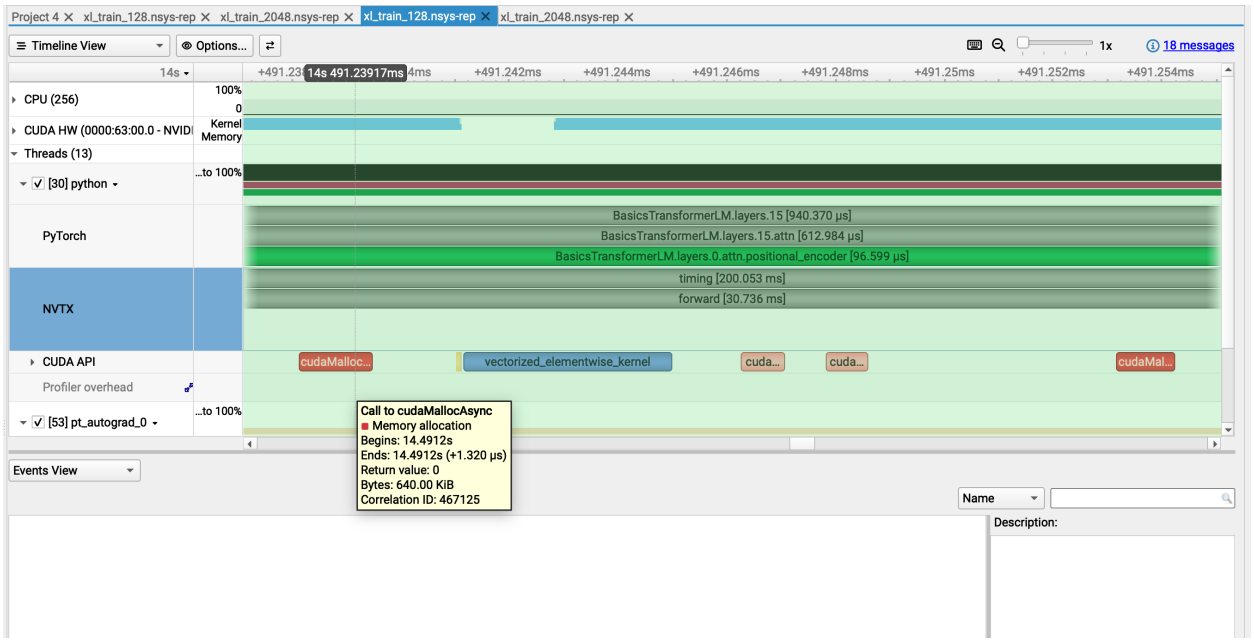
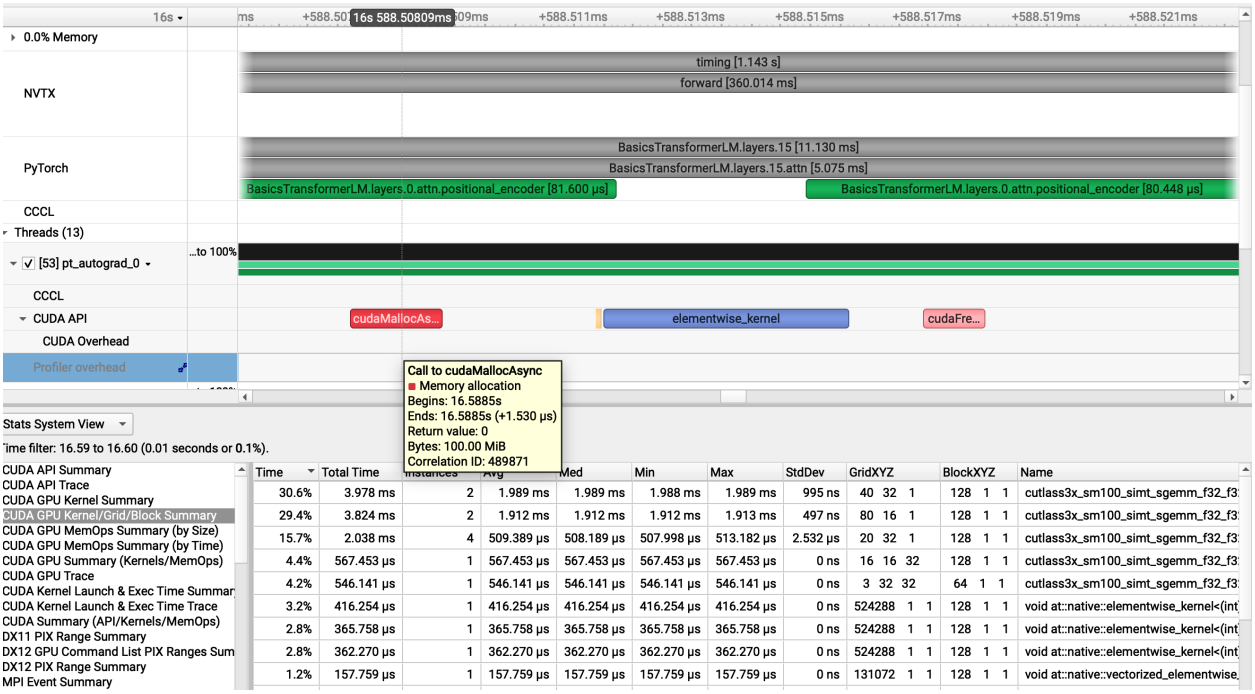


Figure 3: XL model with context length 128 and batch size 1.



Time	Total Time	Med	Min	Max	StdDev	GridXYZ	BlockXYZ	Name
30.6%	3.978 ms	1.989 ms	1.989 ms	1.988 ms	1.989 ms	995 ns	40 32 1	128 1 1	cutlass3x_sm100_simt_sgemm_f32_f3
29.4%	3.824 ms	1.912 ms	1.912 ms	1.912 ms	1.913 ms	497 ns	80 16 1	128 1 1	cutlass3x_sm100_simt_sgemm_f32_f3
15.7%	2.038 ms	509.389 µs	508.189 µs	507.998 µs	513.182 µs	2.532 µs	20 32 1	128 1 1	cutlass3x_sm100_simt_sgemm_f32_f3
4.4%	567.453 µs	567.453 µs	567.453 µs	567.453 µs	567.453 µs	0 ns	16 16 32	128 1 1	cutlass3x_sm100_simt_sgemm_f32_f3
4.2%	546.141 µs	546.141 µs	546.141 µs	546.141 µs	546.141 µs	0 ns	3 32 32	64 1 1	cutlass3x_sm100_simt_sgemm_f32_f3
3.2%	416.254 µs	416.254 µs	416.254 µs	416.254 µs	416.254 µs	0 ns	524288 1 1	128 1 1	void at::native::elementwise_kernel<(int
2.8%	365.758 µs	365.758 µs	365.758 µs	365.758 µs	365.758 µs	0 ns	524288 1 1	128 1 1	void at::native::elementwise_kernel<(int
2.8%	362.270 µs	362.270 µs	362.270 µs	362.270 µs	362.270 µs	0 ns	524288 1 1	128 1 1	void at::native::elementwise_kernel<(int
1.2%	157.759 µs	157.759 µs	157.759 µs	157.759 µs	157.759 µs	0 ns	131072 1 1	128 1 1	void at::native::vectorized_elementwise

Figure 4: XL model with context length 2048 and batch size 1.

At ctx=2048 a single TransformerBlock saves 1.6GiB for the backward pass and the top 5 contributing allocations are all 512 MiB tensors with shape [B, H, seq len, seq len] which is the attention score tensor from the softmax stage. This matches the dominant single allocation which we saw earlier was also from the softmax stage. At ctx=128, the saved size is around 42 MiB and the top contributor is the FFN intermediate tensor with shape [1, 128, 10240] at each around 5 MiB. The attention tensors are only 2 MiB.
The full backward range shows a net memory difference of -39 GiB which is the gradients and activations being freed. Activations are freed across 32 blocks is around 52 GiB, (32 * 1.6 GiB) which makes the gradient memory around 13 GiB, or 407 MiB per TransformerBlock, and this matches hand calculation because we have per block gradients which is 4 attention projections (4 * 2560^2 * 4 = 100 MiB) + the 3 SwiGLU linears (3 * 2560 * 10240 * 4 = 300 MiB), which is around 400 MiB.

3 Single-GPU Memory

3.1 Problem (`gradient_checkpointing`)

(a) Memory-optimal checkpointing strategy

The best strategy with nesting allowed is to recursively store the checkpoints by splitting the N blocks into two halves and wrapping each in a checkpoint, and then within each half doing the same thing as we did on the entire N blocks. This means at any given moment during the backward pass only one boundary tensor per nesting level is stored and since there are $\log_2 N$ levels there is O(log N) peak activation memory.

Additionally, for the compute costs in this scenario, it would be O(N*logN) because each level that is nested has to do a forward pass on the blocks resulting in N * logN work.

```
def forward(transformer_blocks, x):
    if len(transformer_blocks) == 1:
        return transformer_blocks[0](x)
    mid = len(transformer_blocks) // 2
    x = checkpoint(lambda x: forward(transformer_blocks[:mid], x), x, use_reentrant=False)
    x = checkpoint(lambda x: forward(transformer_blocks[mid:], x), x, use_reentrant=False)
    return x
```

(b) Best one-step recomputation strategy for XL

With one level of checkpointing the peak activation memory is roughly

$$M(g) = gT + \frac{N}{g}A,$$

where g is the number of checkpoint groups, T is the residual stream tensor size, N is the number of blocks, and A is the activation memory for one block.

For the XL config at batch=4 and ctx=2048, the residual stream tensor size is

$$T = 4 \cdot 2048 \cdot 2560 \cdot 4 = 83{,}886{,}080 \text{ bytes} \approx 80 \text{ MiB}.$$

From part (f), we can take the per-block saved activation size as about $4 \cdot 1629$ MiB ≈ 6.4 GiB for

batch=4. Since we want to minimize $gT + \frac{N}{g}A$, we get

$$g^\star = \sqrt{\frac{NA}{T}} \approx 50.$$

But since the model only has 32 blocks, we cap this at 32 groups. So the best one-level strategy is just to wrap each TransformerBlock in its own checkpoint, meaning group size 1.

Table 13: Peak GPU memory for the XL model (batch=4, ctx=2048, FP32 train) under single-level gradient checkpointing.

checkpoint group size	peak memory (MiB)
1	65,126
2	71,231
4	83,440

To check this, I ran XL in train mode at batch=4, ctx=2048, FP32 with group sizes 1, 2, and 4. As seen in the table, peak memory increases as group size increases, so the measurements match the prediction that group size 1 is best.

4 GPU Kernels: FlashAttention-2

4.1 Problem (`pytorch_attention`)

Table 14: PyTorch attention benchmark results, batch size 8 and single-head attention.

d_{model}	sequence length	forward time (ms)	backward time (ms)	memory before backward (MiB)
16	256	0.25	0.44	21.21
16	1024	0.25	0.49	84.84
16	4096	2.17	4.34	1070.62
16	8192	8.70	16.60	4205.00
16	16384	33.17	63.96	16713.75
16	32768	OOM	OOM	OOM
32	256	0.26	0.45	22.09
32	1024	0.26	0.50	88.34
32	4096	2.26	4.43	1084.62
32	8192	9.04	16.96	4233.00
32	16384	34.66	65.48	16769.75
32	32768	OOM	OOM	OOM
64	256	0.25	0.46	23.84
64	1024	0.26	0.51	95.34
64	4096	2.58	5.02	1112.62
64	8192	10.28	19.15	4289.00
64	16384	39.71	75.44	16881.75
64	32768	OOM	OOM	OOM
128	256	0.26	0.46	27.34
128	1024	0.28	0.59	109.34
128	4096	3.16	6.14	1168.62
128	8192	12.57	24.00	4401.00
128	16384	48.27	92.04	17105.75
128	32768	OOM	OOM	OOM

Table 15: Smallest PyTorch attention configuration that ran out of memory.

batch size	d_{model}	sequence length	status
8	16	32768	OOM

Memory accounting for seq len = 32768:

Q, K, V: $3 \cdot (8, 32768, 16) \cdot 4 = 48$ MiB. Grad accumulators: 48 MiB. Output: $(8, 32768, 16) \cdot 4 = 16$ MiB. Mask: $(32768, 32768) = 1024$ MiB. Attention scores: $(8, 32768, 32768) \cdot 4 = 32768$ MiB. Attention weights saved: $(8, 32768, 32768) \cdot 4 = 32768$ MiB. So before backward this is already about 65 GiB.

Backward adds the attention weight grad, attention score grad, softmax intermediate, and premask score gradient, each around 32768 MiB. This gives around 195 GiB total, so it OOMs.

Looking at the table, memory grows quadratically with sequence length: every 2× increase in sequence length gives about a 4× increase in memory, which matches the $O(B \cdot S^2)$ attention score and attention weight tensors. The OOM point is also basically independent of d_{model}: at ctx=16384, $d_{\text{model}} = 16$ uses 16713 MiB while $d_{\text{model}} = 128$ uses 17105 MiB, only about a 2.3% difference despite an 8× change in d_{model}, so the S^2 term is dominating.

To eliminate this memory cost, we should tile attention so we never materialize the full $S \times S$ attention matrix. Instead we save only the per-row softmax statistics like the max and sum-exp,

which are $O(B \cdot S)$ instead of $O(B \cdot S^2)$. Then in the backward pass we can just recompute those values.

4.2 Problem (`torch_compile`)

(a) Compiled vs. uncompiled attention

Table 16: Compiled versus uncompiled PyTorch attention, batch size 8 and single-head attention.

d_{model}	seq len	uncompiled fwd (ms)	compiled fwd (ms)	uncompiled bwd (ms)	compiled bwd (ms)	compiled memory (MiB)
16	256	0.25	0.12	0.44	0.20	21.22
16	1024	0.25	0.16	0.49	0.28	84.88
16	4096	2.17	1.04	4.34	1.76	1070.75
16	8192	8.70	3.84	16.60	6.45	4205.25
16	16384	33.17	15.82	63.96	24.00	16714.25
16	32768	OOM	62.87	OOM	108.61	66692.25
32	256	0.26	0.14	0.45	0.21	22.09
32	1024	0.26	0.16	0.50	0.30	88.38
32	4096	2.26	1.38	4.43	2.11	1084.75
32	8192	9.04	5.67	16.96	7.87	4233.25
32	16384	34.66	17.82	65.48	25.59	16770.25
32	32768	OOM	76.74	OOM	117.72	66804.25
64	256	0.25	0.14	0.46	0.20	23.84
64	1024	0.26	0.24	0.51	0.32	95.38
64	4096	2.58	1.70	5.02	2.45	1112.75
64	8192	10.28	6.05	19.15	8.94	4289.25
64	16384	39.71	22.75	75.44	35.43	16882.25
64	32768	OOM	90.23	OOM	147.06	67028.25
128	256	0.26	0.11	0.46	0.14	27.34
128	1024	0.28	0.26	0.59	0.36	109.38
128	4096	3.16	2.23	6.14	3.53	1168.75
128	8192	12.57	8.31	24.00	13.75	4401.25
128	16384	48.27	31.29	92.04	52.01	17106.25
128	32768	OOM	122.00	OOM	211.51	67476.25

(b) Compiled vs uncompiled full Transformer

Table 17: Normal vs. `torch.compile` timing by model size (ctx 512, batch 4, fp32, 15 warmup, 50 iters).

size	fwd uncompiled (ms)	fwd compiled (ms)	bwd uncompiled (ms)	bwd compiled (ms)	opt uncompiled (ms)	opt
small	17.28	13.68	32.60	25.39	11.43	
medium	48.26	40.46	93.34	76.31	20.54	
large	106.86	93.03	209.23	178.47	36.52	
xl	296.86	273.04	573.46	528.47	79.59	

Table 18: Compile speedup relative to normal.

size	forward speedup	backward speedup	optim speedup
small	1.26×	1.28×	1.48×
medium	1.19×	1.22×	0.98×
large	1.15×	1.17×	1.23×
xl	1.09×	1.09×	1.00×

4.3 Problem (`flash_benchmarking`)

(a) Triton FlashAttention vs. PyTorch attention

Table 19: BF16 FlashAttention benchmark timings.

d_{head}	seq len	Triton fwd (ms)	Triton bwd (ms)	Triton e2e (ms)	Torch fwd (ms)	Torch bwd (ms)	Torch e2e (ms)
16	128	0.018	0.324	0.532	0.118	0.673	0.787
16	256	0.018	0.577	0.663	0.119	0.644	0.937
16	512	0.020	0.337	0.604	0.118	0.596	0.851
16	1024	0.027	0.581	0.561	0.119	0.573	1.015
16	2048	0.046	0.431	0.541	0.119	0.542	1.010
16	4096	0.084	0.590	0.676	0.200	0.687	0.946
16	8192	0.159	0.575	0.721	0.826	1.468	2.280
16	16384	0.314	1.929	2.239	2.982	5.352	8.313
16	32768	0.914	7.595	8.504	11.433	20.807	32.212
16	65536	3.572	30.678	34.259	45.364	82.845	128.206
32	128	0.018	0.506	0.764	0.122	0.735	1.033
32	256	0.017	0.606	0.782	0.123	0.719	0.991
32	512	0.020	0.609	0.674	0.121	0.578	1.021
32	1024	0.021	0.614	0.820	0.123	0.633	1.015
32	2048	0.028	0.637	0.729	0.115	0.549	0.999
32	4096	0.048	0.590	0.631	0.200	0.642	0.944
32	8192	0.088	0.777	0.859	0.825	1.455	2.266
32	16384	0.222	2.972	3.188	2.980	5.344	8.303
32	32768	0.767	11.773	12.539	11.435	20.781	32.199
32	65536	2.607	46.057	48.669	45.364	82.877	128.157
64	128	0.019	0.626	0.746	0.119	0.620	0.965
64	256	0.018	0.586	0.705	0.119	0.522	1.000
64	512	0.020	0.598	0.648	0.124	0.758	0.960
64	1024	0.022	0.650	0.648	0.120	0.709	0.975
64	2048	0.031	0.540	0.636	0.125	0.632	1.005
64	4096	0.053	0.681	0.726	0.200	0.581	1.018
64	8192	0.098	0.763	0.856	0.824	1.453	2.264
64	16384	0.255	2.962	3.215	2.986	5.365	8.334
64	32768	0.953	11.617	12.572	11.437	20.810	32.244
64	65536	3.112	46.246	49.390	45.360	82.845	128.114
128	128	0.020	0.578	0.699	0.127	0.605	0.943
128	256	0.020	0.657	0.603	0.125	0.680	0.991
128	512	0.022	0.583	0.726	0.123	0.673	0.967
128	1024	0.032	0.679	0.796	0.123	0.574	0.964
128	2048	0.057	0.502	0.722	0.124	0.632	1.002
128	4096	0.106	0.611	0.673	0.203	0.676	1.023
128	8192	0.276	0.778	1.049	0.829	1.462	2.276
128	16384	0.969	2.962	3.926	3.002	5.379	8.358
128	32768	3.384	11.685	15.061	11.503	20.858	32.339
128	65536	13.352	46.841	60.196	45.747	83.241	128.929

Table 20: FP32 FlashAttention benchmark timings.

d_{head}	seq len	Triton fwd (ms)	Triton bwd (ms)	Triton e2e (ms)	Torch fwd (ms)	Torch bwd (ms)	Torch e2e (ms)
16	128	0.018	0.569	0.747	0.129	0.700	1.060
16	256	0.022	0.319	0.697	0.128	0.525	0.880
16	512	0.030	0.666	0.761	0.130	0.662	1.025
16	1024	0.052	0.572	0.602	0.128	0.778	0.937
16	2048	0.094	0.404	0.694	0.128	0.700	1.077
16	4096	0.181	0.593	0.794	0.385	0.763	1.150
16	8192	0.357	1.399	1.759	1.256	2.461	3.712
16	16384	0.730	4.791	5.525	4.349	8.681	13.013
16	32768	2.902	19.522	22.417	17.296	34.369	51.658
16	65536	10.858	73.303	84.165	67.256	131.520	198.727
32	128	0.020	0.572	0.567	0.128	0.698	0.801
32	256	0.019	0.589	0.625	0.127	0.692	0.925
32	512	0.022	0.639	0.789	0.125	0.755	1.036
32	1024	0.023	0.366	0.650	0.128	0.613	1.035
32	2048	0.036	0.573	0.739	0.134	0.566	0.953
32	4096	0.063	0.609	0.623	0.399	0.731	1.124
32	8192	0.117	1.501	1.616	1.308	2.451	3.757
32	16384	0.303	5.257	5.557	4.610	8.805	13.405
32	32768	1.151	20.727	21.880	17.986	34.575	52.551
32	65536	3.704	79.127	82.843	70.143	134.480	204.545
64	128	0.019	0.497	0.776	0.123	0.673	1.035
64	256	0.020	0.659	0.701	0.129	0.751	0.752
64	512	0.023	0.546	0.777	0.126	0.749	0.930
64	1024	0.032	0.574	0.756	0.129	0.713	0.970
64	2048	0.055	0.638	0.792	0.138	0.593	0.946
64	4096	0.101	0.557	0.754	0.417	0.792	1.206
64	8192	0.193	1.809	1.999	1.409	2.679	4.074
64	16384	0.759	6.328	7.086	5.025	9.609	14.626
64	32768	2.959	24.832	27.787	19.598	37.669	57.271
64	65536	11.732	97.994	109.728	77.647	149.003	226.609
128	128	0.020	0.653	0.706	0.127	0.712	1.045
128	256	0.019	0.432	0.780	0.127	0.584	1.029
128	512	0.025	0.554	0.630	0.127	0.690	0.762
128	1024	0.042	0.631	0.538	0.131	0.703	1.069
128	2048	0.076	0.584	0.643	0.156	0.687	0.948
128	4096	0.144	0.712	0.856	0.458	0.921	1.373
128	8192	0.414	2.387	2.798	1.641	3.147	4.785
128	16384	1.629	8.996	10.622	6.105	11.760	17.859
128	32768	6.460	35.482	41.936	23.893	46.245	70.141
128	65536	22.587	143.200	165.787	93.658	188.025	279.412

5 Distributed Data Parallel Training

5.1 Problem (`distributed_communication_single_node`)

Looking at the data we see that due to the fixed overhead that is paid from kernel launches, NCCL setups, and other workload setups, at 1MB adding ranks doesn't speedup anything and change wall clock time. Furthermore, we can see at 10MB, N=4 actually outperforms N=6 at effective bandwidth, but overall trends show that scaling number of ranks raises effective bandwidth, but it seems to be plateauing.

Table 21: All-reduce timing and effective bandwidth

	time (ms)			bandwidth (GB/s)		
size (MB)	N=2	N=4	N=6	N=2	N=4	N=6
1	0.055	0.040	0.048	18.9	39.2	36.4
10	0.078	0.076	0.124	134.9	207.9	141.1
100	0.268	0.304	0.318	391.0	517.4	549.8
1000	1.892	2.563	2.782	554.2	613.7	628.2

5.2 Problem (`naive_ddp_benchmarking`)

I benchmarked the naive DDP implementation with two B200 GPUs using NCCL. Context length 512, batch size 1 per rank, 5 warmup steps, and 10 measured training iterations. I recorded communication time as time spent in the post-backward per-parameter gradient all-reduces.

Table 22: Naive DDP benchmark for the XL model on 1 node × 2 GPUs.

model	global batch	step time (ms)	comm time (ms)	comm fraction	peak memory (MiB)
xl	2	402.24 ± 0.38	36.13 ± 0.20	8.98%	52649.80

The naive implementation spends about 36.13 ms per iteration on communication, which is 8.98% of the total 402.24 ms training step.

5.3 Problem (`minimal_ddp_flat_benchmarking`)

Table 23: Flat-gradient DDP benchmark for the XL model on 1 node × 2 GPUs.

model	global batch	step time (ms)	comm time (ms)	comm fraction	peak memory (MiB)
xl	2	403.85 ± 0.70	37.15 ± 0.23	9.20%	65145.11

As we see in the data, flattening gradients did not improve performance: communication time increased slightly from 36.13 ms to 37.15 ms, and total step time increased from 402.24 ms to 403.85 ms. This suggests that for this setup the extra flatten/unflatten work outweighed the benefit from reducing the number of all-reduce calls.

5.4 Problem (`ddp_overlap_individual_parameters_benchmarking`)

(a) Time per training iteration with overlap

Table 24: Overlapped individual-parameter DDP benchmark for the XL model on 1 node × 2 GPUs.

model	global batch	step time (ms)	exposed sync time (ms)	sync fraction	peak memory (MiB)
xl	2	380.10 ± 0.91	1.13 ± 0.28	0.30%	52649.80

Overlapping individual gradient all-reduces with backward computation reduced total step time from 402.24 ms for the non-overlapped per-parameter version to 380.10 ms, and the post-backward synchronization time dropped from 36.13 ms to 1.13 ms because a lot of it was covered. This is also faster than the flat-gradient version, which took 403.85 ms per step.

(b) Nsight comparison screenshots

As we can see in the figures below, NCCL does not run during backward pass of no DDP, but then with DDP, there is communication happening during the backward pass as we can see in the screenshot.

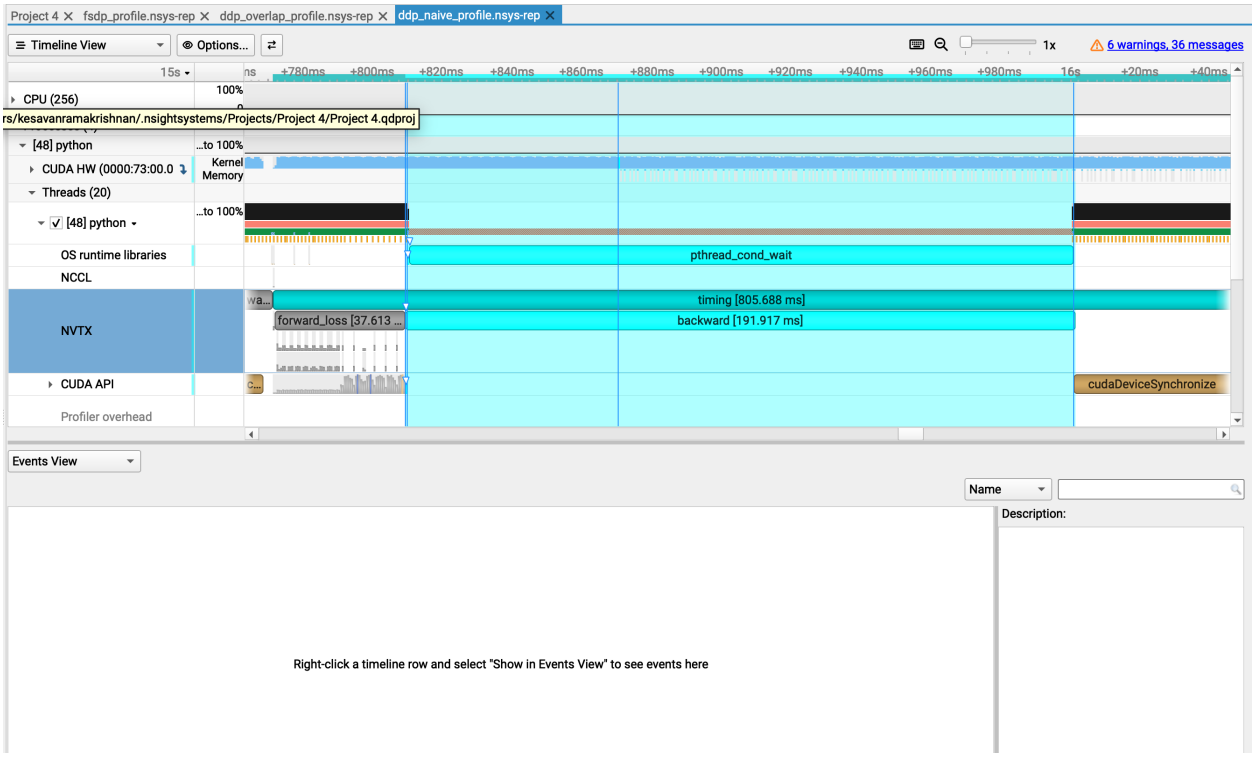


Figure 5: Nsight Systems trace for DDP backward pass with no comms.

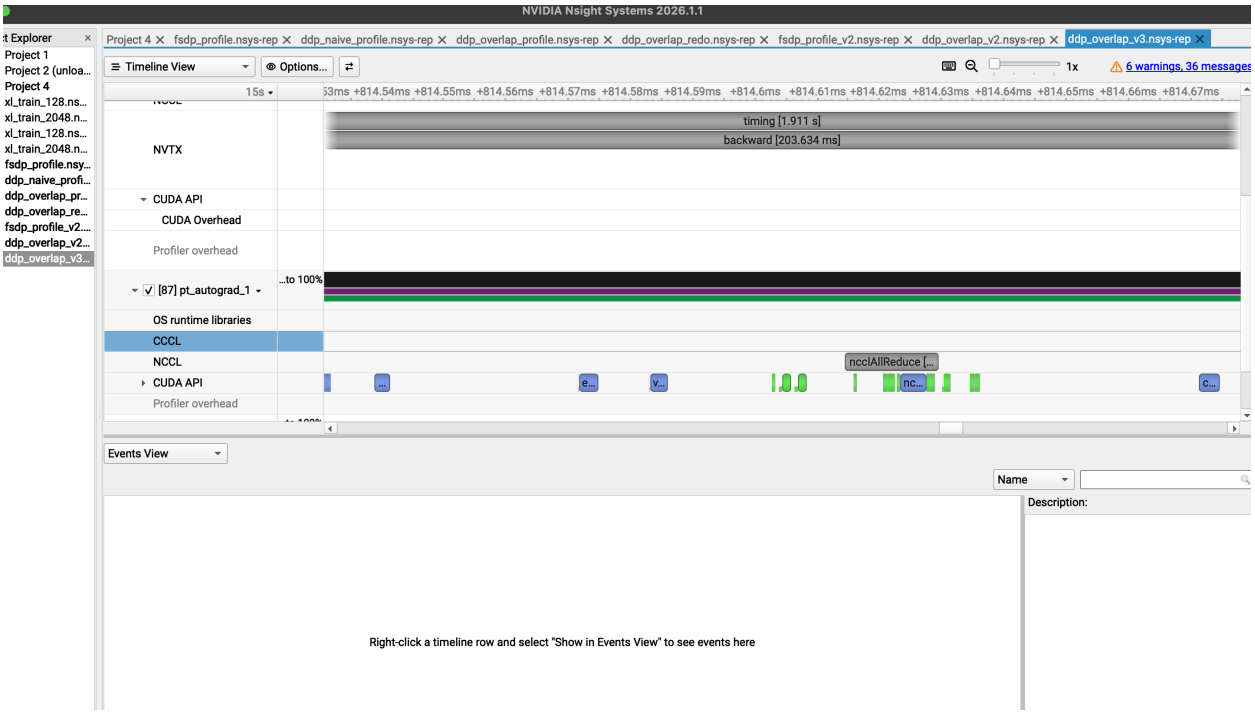


Figure 6: Nsight Systems trace for DDP backward pass with no comms.

6 Optimizer State Sharding

6.1 Problem (`optimizer_state_sharding_accounting`)

(a) Peak memory with vs. without sharding

Table 25: Optimizer state sharding memory usage for the XL model on 1 node × 2 GPUs.

optimizer	after model init (MiB)	before optim step (MiB)	after optim step (MiB)	peak (MiB)
unsharded	13124.79	52149.11	52149.11	65145.11
sharded	13124.79	39348.12	39348.12	52344.47

With optimizer sharding the state reduced peak memory from 65145.11 MiB to 52344.47 MiB. Model parameters are unchanged after initialization, but after gradients and optimizer state are materialized the sharded optimizer uses about 12800.99 MiB less memory per rank.

From looking at the data we can see that parameters accounted for around 13GB, from after model init, gradients were also therefore 13GB, and optimizer state accounts for about 26 GB, since theres 32 GB for params + gradients, and 26 GB leftover for the unsharded optimizer right before the optim step, but 13 GB per rank when world size is 2.

(b) Effect of sharding on training speed

Table 26: Optimizer state sharding timing for the XL model on 1 node × 2 GPUs.

optimizer	iteration time (ms)	optimizer step (ms)	global batch	context length
unsharded	401.58 ± 0.32	79.98 ± 0.15	2	512
sharded	391.38 ± 0.43	69.66 ± 0.05	2	512

Looking at the data we can see that using optimizer sharding resulted in lower training step times. The iteration time was about 10ms faster than unsharded, coming from the 10ms speedup in the optimizer step.

(c) Comparison to ZeRO stage 1

Our implementation shards optimizer state which is also what ZeRO does, but still stores the full gradient on every rank after the all-reduce, and synchronizes parameters with one broadcast per parameter from the owning rank. ZeRO uses a reduce-scatter primitive for gradients (so each rank only retains its own slice — saving roughly P * (N-1)/N of memory per rank from the graidents) and an all-gather to bring back parameters after the step. Because of this, ZeRO-1 has both lower per-rank memory and lower total communication volume (2P bytes vs. 3P for our implementation).

7 Fully-Sharded Data Parallel

7.1 Problem (`fsdp_accounting`)

(a) Expected memory savings from FSDP

Without FSDP, we have, per rank, peak memory usage = 4W + A (where W is size per rank for parameters) and A is activation size. With FSDP, we end up with 4W/N + A (where N is number of ranks), and if we were to plug in XL numbers where memory after model init was 13124.79 MiB, we can use this as W, and at peak it was 65145.11 MiB, and see that with FSDP we would then theoretically have $65145.11 - 2 * W = 38895$ MiB, meaning our savings was 2 * W $\approx$ **26250 MiB**.

(b) All-gather overlap with the forward pass

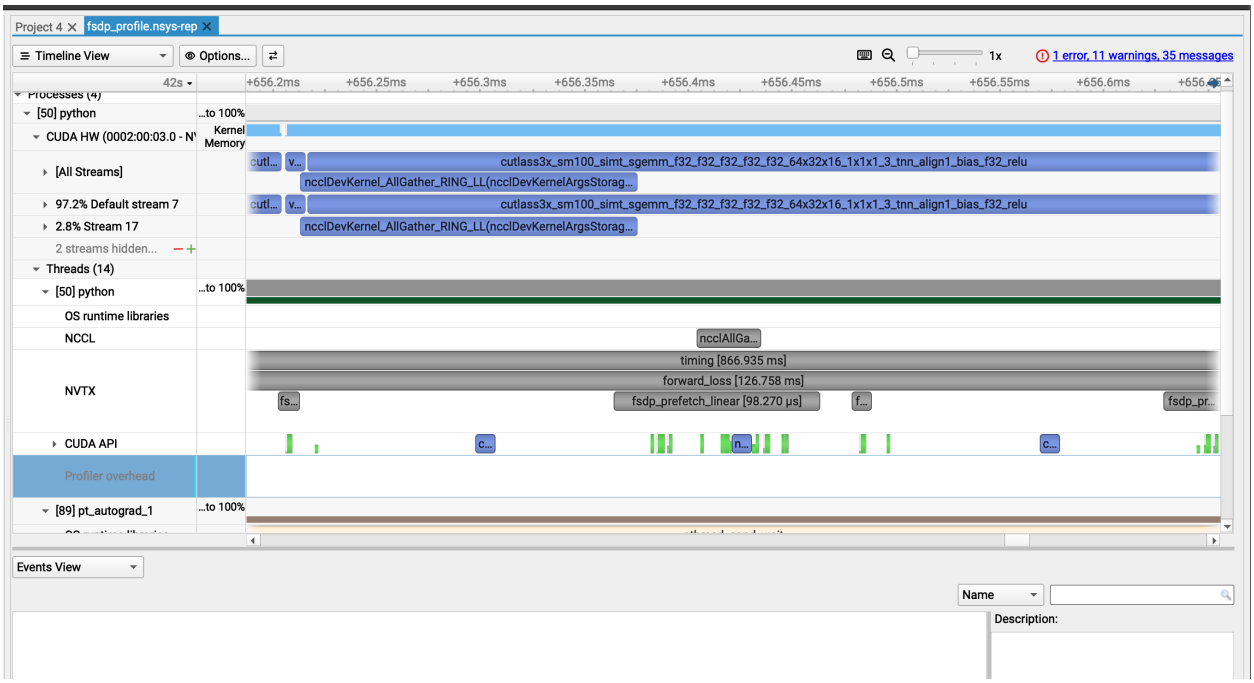


Figure 7: Nsight Systems trace for FSDP on the XL model with 2 GPUs.

`cutlass3x_sm100_simt_sgemm` kernels on the default compute stream while `ncclDevKernel_AllGather_RING_LL` runs at the same time on a separate NCCL stream. A single all-gather overlaps with two consecutive layer matmuls and this makes sense with the prefetch lookahead of 2: the all-gather for layer $i + 2$ starts during layer i's compute and finishes before layer $i + 2$ calls `handle.wait()` in its forward pass. The CPU-side prefetch range returns quickly, showing that prefetch only launches the asynchronous collective and does not wait for the communication to finish.

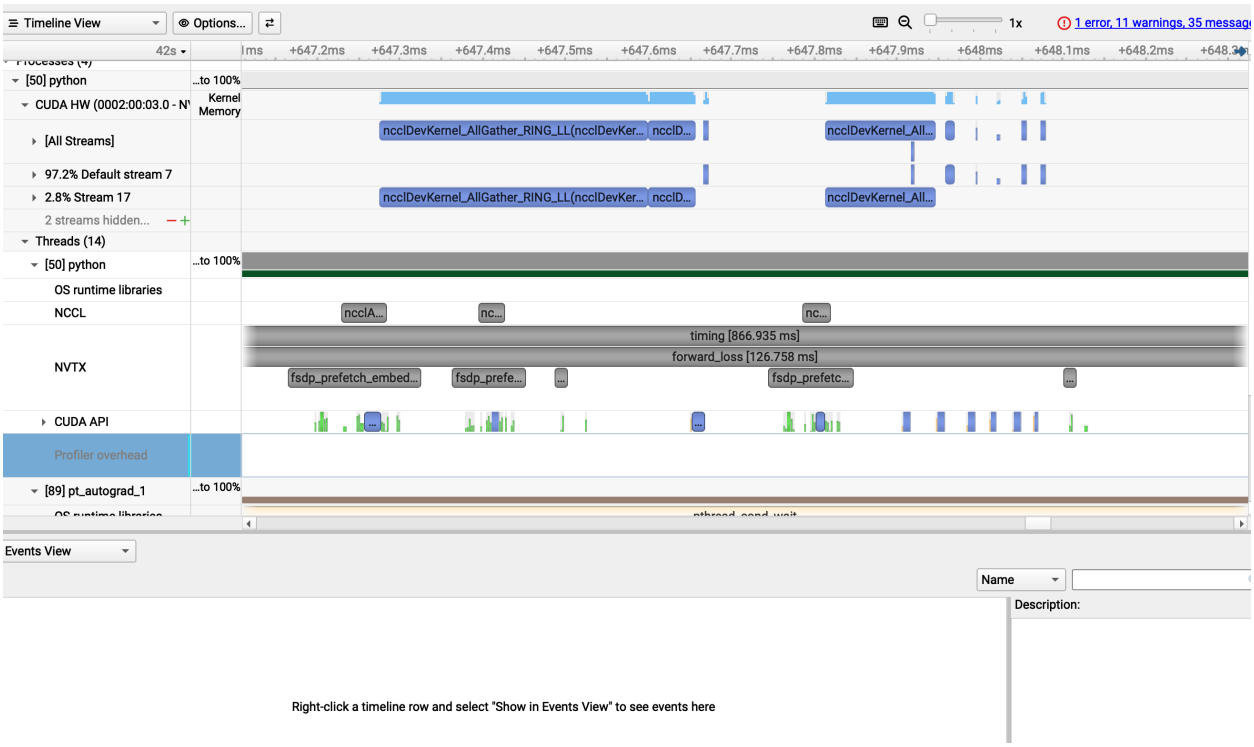


Figure 8: Nsight Systems trace for FSDP for first two layers showing no overlaps.

However, looking at the first two layers we see no overlap of compute and comms, which makes sense because the layer's haven't been prefetched yet so the first two are done concurrently.

8 Analyzing Parallelism Strategies

8.1 Problem (`alternate_ring_all_reduce`)

This would take $\frac{S*(N-1)}{W}$/ seconds because for each step there is a tensor of size S transmitted with N - 1 steps and W bandwidth.

8.2 Problem (`data_parallel_calcs`)

(a) Backward FLOPs with $N_{\mathbf{DP}}$

For the FFN backward pass, the per-device FLOP count is

$$\frac{12BDD_{\mathrm{FF}}}{N_{\mathrm{DP}}}.$$

This comes from teh 6 matmuls:

$$dz = dyW_3^T, \quad dx_1 = dx_1W_1^T, \quad dx_2 = dx_2W_2^T, \quad dW_3 = z^T dy, \quad dW_2 = x^T dx_2, \quad dW_1 = x^T dx_1.$$

Each is $2BDD_{\mathrm{FF}}$ in terms of FLOPs $6 \cdot 2BDD_{\mathrm{FF}} = 12BDD_{\mathrm{FF}}$. Since each device only processes B/N_{DP} rows

$$\frac{\mathbf{12BDD_{FF}}}{\mathbf{N_{DP}}}.$$

(b) Backward communication time with $N_{\mathbf{DP}}$

The backward pass comm time is

$$\mathbf{T}_{\mathrm{comm}} = \frac{\mathbf{12DD_{FF}}}{\mathbf{W}} \cdot \frac{\mathbf{N_{DP}-1}}{\mathbf{N_{DP}}}.$$

We have to all-reduce the weight gradients dW_1, dW_2, dW_3, which together have $3DD_{\mathrm{FF}}$ FP16 elements -¿ $6DD_{\mathrm{FF}}$ bytes.

(c) Maximum $N_{\mathbf{DP}}$ before bottleneck

Compute time per device is

$$T_{\mathrm{comp}} = \frac{12BDD_{\mathrm{FF}}}{N_{\mathrm{DP}}C}.$$

Communication becomes the bottleneck when

$$T_{\text{comm}} > T_{\text{comp}}.$$

$$\frac{12DD_{\text{FF}}}{W} \cdot \frac{N_{\text{DP}} - 1}{N_{\text{DP}}} > \frac{12BDD_{\text{FF}}}{N_{\text{DP}}C}$$

so

$$\mathbf{N_{\text{DP}} > 1 + \frac{BW}{C}}$$

Beause of this DP is not communication bottlenecked as long as

$$\mathbf{N_{\text{DP}} \leq 1 + \frac{BW}{C}}$$

8.3 Problem (`fsdp_calcs`)

(a) FLOPs for forward and backward

Backward pass:

$$\mathbf{\frac{12BDD_{\text{FF}}}{N_{\text{FSDP}}}}$$

FLOPs per device.

This is the same as DP because each device computes backward on batch size B/N_{FSDP}.

Forward pass:

$$\mathbf{\frac{6BDD_{\text{FF}}}{N_{\text{FSDP}}}}$$

FLOPs per device.

This is because the forward pass has three matmuls, each with batch size B/N_{FSDP}.

(b) Communication time for forward and backward

Backward communication time:

$$\mathbf{T_{\text{comm,bwd}} = \frac{12DD_{\text{FF}}}{W} \cdot \frac{N_{\text{FSDP}} - 1}{N_{\text{FSDP}}}}$$

This comes from all gather of three weight matrices and reduce scatter of the three weight gradients.

Forward communication time:

$$\mathbf{T}_{\text{comm,fwd}} = \frac{\mathbf{6DD}_{\text{FF}}}{\mathbf{W}} \cdot \frac{\mathbf{N}_{\text{FSDP}} - \mathbf{1}}{\mathbf{N}_{\text{FSDP}}}$$

This comes from the all gather on the three weight matrices that have to be done before using them.

(c) Maximum N_{FSDP} before bottleneck

Backward compute time is

$$T_{\text{comp,bwd}} = \frac{12BDD_{\text{FF}}}{N_{\text{FSDP}}C}.$$

Communication becomes the bottleneck when

$$T_{\text{comm,bwd}} > T_{\text{comp,bwd}}.$$

$$\frac{12DD_{\text{FF}}}{W} \cdot \frac{N_{\text{FSDP}} - 1}{N_{\text{FSDP}}} > \frac{12BDD_{\text{FF}}}{N_{\text{FSDP}}C}$$

so

$$\mathbf{N}_{\text{FSDP}} > \mathbf{1} + \frac{\mathbf{BW}}{\mathbf{C}}$$

FSDP is not communication bottlenecked when

$$\mathbf{N}_{\text{FSDP}} \leq \mathbf{1} + \frac{\mathbf{BW}}{\mathbf{C}}$$

Forward compute time is

$$T_{\text{comp,fwd}} = \frac{6BDD_{\text{FF}}}{N_{\text{FSDP}}C}.$$

Comms becomes the bottleneck when

$$T_{\text{comm,fwd}} > T_{\text{comp,fwd}}.$$

$$\frac{6DD_{\text{FF}}}{W} \cdot \frac{N_{\text{FSDP}} - 1}{N_{\text{FSDP}}} > \frac{6BDD_{\text{FF}}}{N_{\text{FSDP}}C}$$

so

$$\mathbf{N}_{\text{FSDP}} > \mathbf{1} + \frac{\mathbf{BW}}{\mathbf{C}}$$

Forward FSDP is not communication bottlenecked as long as

$$\mathbf{N}_{\text{FSDP}} \leq \mathbf{1} + \frac{\mathbf{BW}}{\mathbf{C}}$$

8.4 Problem (`tp_calcs`)

(a) Backward pass equations for TP

The TP backward pass is

$$dz^{(i)} = dy \left(W_3^{(i)}\right)^\top$$

$$dx_2^{(i)} = dz^{(i)} * f(x_1^{(i)})$$

$$dx_1^{(i)} = dz^{(i)} * f'(x_1^{(i)}) * x_2^{(i)}$$

$$dW_3^{(i)} = \left(z^{(i)}\right)^\top dy$$

$$dW_2^{(i)} = x^\top dx_2^{(i)}$$

$$dW_1^{(i)} = x^\top dx_1^{(i)}$$

$$dx_{\text{partial}}^{(i)} = dx_1^{(i)} \left(W_1^{(i)}\right)^\top + dx_2^{(i)} \left(W_2^{(i)}\right)^\top$$

$$dx = \text{all-reduce}\left(\left\{dx_{\text{partial}}^{(i)}\right\}_{i=0}^{N_{\text{TP}}-1}\right)$$

(b) FLOPs for forward and backward with N_{TP}

Forward pass:

$$\frac{\mathbf{6BDD}_{\text{FF}}}{\mathbf{N}_{\text{TP}}}$$

FLOPs per device.

This is because there are three forward matmuls, each sharded by N_{TP}.

Backward pass:

$$\frac{\mathbf{12BDD_{FF}}}{\mathbf{N_{TP}}}$$

FLOPs per device.

This is because there are six backward matmuls, each sharded by N_{TP}.

(c) Communication time for forward and backward

Forward communication time:

$$\mathbf{T_{comm,fwd}} = \frac{\mathbf{4BD}}{\mathbf{W}} \cdot \frac{\mathbf{N_{TP} - 1}}{\mathbf{N_{TP}}}$$

This comes from the all-reduce from $y^{(i)}$, which has shape (B, D).

Backward communication time:

$$\mathbf{T_{comm,bwd}} = \frac{\mathbf{4BD}}{\mathbf{W}} \cdot \frac{\mathbf{N_{TP} - 1}}{\mathbf{N_{TP}}}$$

This comes from the all-reduce from $dx^{(i)}_{\text{partial}}$, that has shape (B, D).

(d) Maximum N_{TP} before bottleneck

Forward compute time is

$$T_{\text{comp,fwd}} = \frac{6BDD_{\text{FF}}}{N_{\text{TP}}C}.$$

Communication becomes the bottleneck when

$$T_{\text{comm,fwd}} > T_{\text{comp,fwd}}.$$

$$\frac{4BD}{W} \cdot \frac{N_{\text{TP}} - 1}{N_{\text{TP}}} > \frac{6BDD_{\text{FF}}}{N_{\text{TP}}C}$$

so

$$\mathbf{N_{TP}} > \mathbf{1} + \frac{\mathbf{3D_{FF}W}}{\mathbf{2C}}$$

Forward TP is not comm bottlenecked as long as

$$\mathbf{N_{TP}} \leq \mathbf{1} + \frac{\mathbf{3D_{FF}W}}{\mathbf{2C}}$$

Backward compute time is

$$T_{\text{comp,bwd}} = \frac{12BDD_{\text{FF}}}{N_{\text{TP}}C}.$$

Comms becomes the bottleneck when

$$T_{\text{comm,bwd}} > T_{\text{comp,bwd}}.$$

$$\frac{4BD}{W} \cdot \frac{N_{\text{TP}} - 1}{N_{\text{TP}}} > \frac{12BDD_{\text{FF}}}{N_{\text{TP}}C}$$

so

$$\mathbf{N_{TP} > 1 + \frac{3D_{FF}W}{C}}$$

Backward TP is not communication bottlenecked as long as

$$\mathbf{N_{TP} \leq 1 + \frac{3D_{FF}W}{C}}$$

8.5 Problem (`fsdp_tp_calcs`)

(a) Forward FLOPs for FSDP + TP

Forward pass:

$$\mathbf{\frac{6BDD_{FF}}{N_{FSDP}N_{TP}}}$$

FLOPs per rank.

This is because the batch is sharded by N_{FSDP}, and each matmul is sharded by N_{TP}.

(b) Forward communication time (overlapping axes)

The FSDP communication cost is

$$\frac{6DD_{\text{FF}}}{N_{\text{TP}}W} \cdot \frac{N_{\text{FSDP}} - 1}{N_{\text{FSDP}}}.$$

The TP communication cost is

$$\frac{4BD}{N_{\text{FSDP}}W} \cdot \frac{N_{\text{TP}} - 1}{N_{\text{TP}}}.$$

Since the two both can overlap,

$$\mathbf{T}_{\text{comm,fwd}} = \max\left(\mathbf{\frac{6DD_{FF}}{N_{TP}W}} \cdot \mathbf{\frac{N_{FSDP} - 1}{N_{FSDP}}}, \mathbf{\frac{4BD}{N_{FSDP}W}} \cdot \mathbf{\frac{N_{TP} - 1}{N_{TP}}}\right)$$

This comes from FSDP all-gathering the weights and TP all-reducing the output activations.

(c) Maximum N with overlap

Compute time is

$$T_{\text{comp,fwd}} = \frac{6BDD_{\text{FF}}}{N_{\text{FSDP}}N_{\text{TP}}C}.$$

To avoid ommunication bottleneck, both comm costs must be at most compute time.

For FSDP:

$$\frac{6DD_{\text{FF}}}{N_{\text{TP}}W} \cdot \frac{N_{\text{FSDP}} - 1}{N_{\text{FSDP}}} \leq \frac{6BDD_{\text{FF}}}{N_{\text{FSDP}}N_{\text{TP}}C}$$

so

$$N_{\text{FSDP}} \leq 1 + \frac{BW}{C}.$$

For TP:

$$\frac{4BD}{N_{\text{FSDP}}W} \cdot \frac{N_{\text{TP}} - 1}{N_{\text{TP}}} \leq \frac{6BDD_{\text{FF}}}{N_{\text{FSDP}}N_{\text{TP}}C}$$

so

$$N_{\text{TP}} \leq 1 + \frac{3D_{\text{FF}}W}{2C}.$$

So best would be

$$\mathbf{N} = \mathbf{N}_{\text{FSDP}}\mathbf{N}_{\text{TP}} \leq \left(\mathbf{1} + \frac{\mathbf{BW}}{\mathbf{C}}\right)\left(\mathbf{1} + \frac{\mathbf{3D}_{\text{FF}}\mathbf{W}}{\mathbf{2C}}\right)$$

(d) Maximum N without overlap

If the collectives cannot overlap, then

$$T_{\text{comm,fwd}} = \frac{6DD_{\text{FF}}}{N_{\text{TP}}W} \cdot \frac{N_{\text{FSDP}} - 1}{N_{\text{FSDP}}} + \frac{4BD}{N_{\text{FSDP}}W} \cdot \frac{N_{\text{TP}} - 1}{N_{\text{TP}}}.$$

We need

$$T_{\text{comm,fwd}} \leq T_{\text{comp,fwd}}.$$

We then have

$$3D_{\mathrm{FF}}(N_{\mathrm{FSDP}}-1)+2B(N_{\mathrm{TP}}-1)\leq\frac{3BD_{\mathrm{FF}}W}{C}.$$

If we use

$$A=3D_{\mathrm{FF}},\qquad Q=2B,\qquad K=\frac{3BD_{\mathrm{FF}}W}{C}.$$

Then we maximize

$$N=N_{\mathrm{FSDP}}N_{\mathrm{TP}}$$

wrt

$$A(N_{\mathrm{FSDP}}-1)+Q(N_{\mathrm{TP}}-1)\leq K.$$

The optimal setting gets us

$$N_{\mathrm{FSDP}}=\frac{K+A+Q}{2A},\qquad N_{\mathrm{TP}}=\frac{K+A+Q}{2Q}.$$

Thus,

$$\mathbf{N}\leq\frac{\left(\frac{\mathbf{3BD_{FF}W}}{\mathbf{C}}+\mathbf{3D_{FF}}+\mathbf{2B}\right)^{\mathbf{2}}}{\mathbf{24BD_{FF}}}$$

9 Leaderboard

9.1 Problem (`leaderboard`)

For the leaderboard I implemented the following:

- CuTile Python Flash attention fwd + bwd - Causal masking optimizations - FSDP - Checkpoint every 4 layers

Ended with 4207 ms/step on 2xB200.